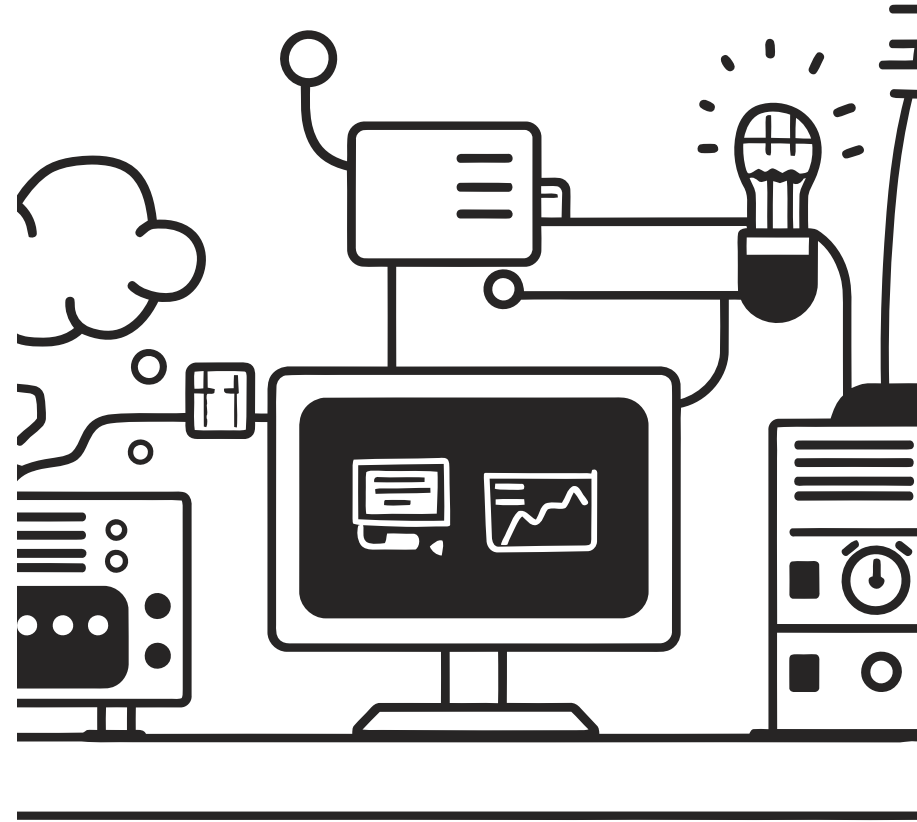


Applying LLM Capabilities For Group Projects

Bernhard Krüpl-Sypien
bernhard@became.ai

05 November 2025



Modes of Personal Knowledge Management

We propose the following mental model of modes when working with PKM:

- **Collect**
 - Collect data / information (/ knowledge?)
 - Traditional challenges: Information overload, inconsistent formats
- **Surface**
 - Find collected thoughts, either active (search) or passive (agentic)
- **Connect**
 - Connect related ideas, consider perspective
- **Synthesize**
 - Synthesize knowledge from collection
- **Reflect**
 - Analyze thinking patterns and knowledge gaps

Types of Pre-Trained Models

1 Foundation Models (= Base Models)

Usually trained for next token prediction / text completion

2 Fine-Tuned Models

Specialised for specific tasks:

- *Instruct models*
 - follow instructions on a single prompt
 - often work well in conversations
- *Chat models*
 - Optimised for multi-turn dialogues and maintaining context over several interactions
- *Multi-modal models*
 - Combine information from different modalities such as text with images etc.

3 Embedding Models

Generate dense vector representations of input data for tasks like similarity search, clustering, or recommendation systems. Often trained with objectives like contrastive learning or triplet loss to enhance semantic relationships in the embedding space.

Foundation of LLM Behaviour

The output of an LLM relies on its *weights* and *context* – and the *program* running it ("inference")

- **Weights (long-term memory)**
 - Learnt by original training of the model
 - Determine how the model processes input and produces output
- **Context (short-term memory)**
 - The text provided to the model at inference time – system prompt, user prompt, chat history, and any additional information as in RAG
 - This short-term memory is filled with previous interactions
 - But is also the room for future interactions
 - Models with large context windows tend to forget the middle of the context

Prompting for In-Context Adaptation

Large Language Models (LLMs) can adapt to new tasks from examples provided directly within the prompt, a technique known as In-Context Learning. This method allows for rapid task adaptation without requiring expensive re-training or fine-tuning. We prefer to call it In-Context *Adaptation* instead of *Learning*, because there are no parameters updated or learnt.

Zero-Shot Prompting

No examples are provided in the prompt. The LLM relies solely on its vast pre-trained knowledge to understand the instruction and perform the task.

Query

Classify the sentiment of the following text as positive, negative, or neutral.

Text: I liked the hotel.

Sentiment:

Answer

positive

One-Shot Prompting

A single example is included in the prompt. This provides a clear template, helping the model understand the desired output format or style for a specific task.

Query

Classify the sentiment of the following text as positive, negative, or neutral.

Text: The hotel is terrible.

Sentiment: negative

Text: I think the food was okay. Sentiment:

Answer

neutral

Few-Shot Prompting

Multiple examples are supplied in the prompt. This significantly enhances the model's ability to follow complex instructions and quickly adapt to novel tasks.

Query

Classify the sentiment of the following text as positive, negative, or neutral.

Text: The hotel is terrible.

Sentiment: negative

Text: Excellent service, appreciated it
Sentiment: positive

Text: The bed was dirty and too short.
Sentiment:

Answer

negative

Fine-Tuning

Fine-tuning takes a pre-trained model and trains it further on a smaller, task-specific dataset. Unlike in-context learning, which leverages examples within a prompt, fine-tuning modifies the model's parameters directly.

Usually **GOOD** for focusing and formatting.

Significantly improves accuracy and quality on specific target tasks.

Tailors the model's understanding and generation to a particular field.

Usually **BAD** for adding new knowledge due to overfitting and catastrophic forgetting – use RAG instead

Creating LLM Flows

- Chains of operations often work better than zero-shot
- Breaking complex tasks into manageable steps
- Example: Multiple LLM calls for summarise → extract → format instead of one single step that tries to do all at once

❏ **Implementation:** Multi-step composition in code (cmp. chain mini pattern). Orchestration frameworks like LangChain. Flow engineering tools.

Common Architecture Patterns



Context & History Handling

Managing conversation state, summarizing long contexts, and strategically pruning history to stay within token limits whilst preserving critical information.



Data Formats

Structured input/output handling through JSON schemas, XML templates, or custom formats ensuring reliable parsing and validation of model responses.



Workflows & Chains

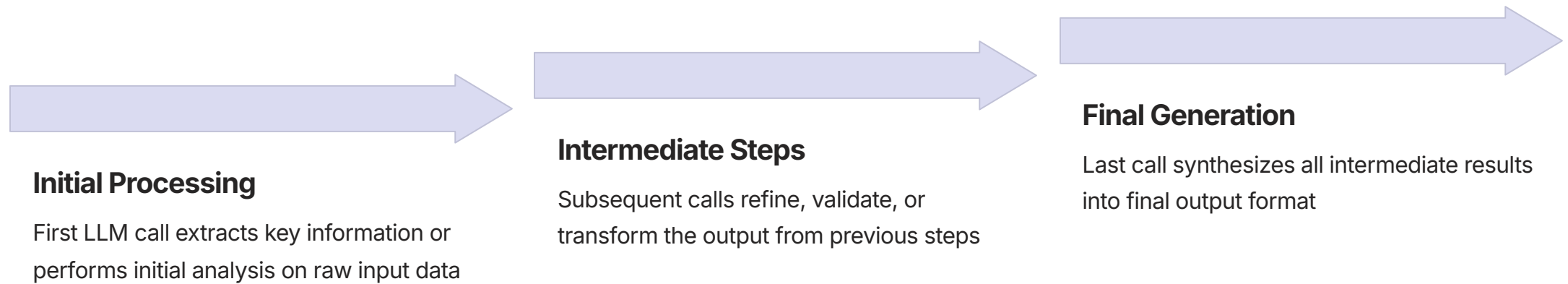
Connecting multiple LLM calls in sequence, passing outputs as inputs to subsequent steps, enabling complex multi-stage processing and reasoning.



Tool Usage & Function Calling

Calling external tools and functions to augment context.

Mini-Pattern: Multi-Step Processing



Building Simple Chains

```
def process_chain(text):  
    # Step 1: Summarize text  
    summary = llm_call(  
        f"Summarize: {text}"  
    )  
  
    # Step 2: Extract actions  
    actions = llm_call(  
        f"Extract actions from: {summary}"  
    )  
  
    # Step 3: Format output  
    return llm_call(  
        f"Prioritize: {actions}"  
    )
```